

FragPicker: A New Defragmentation Tool for Modern Storage Devices

Jonggyu Park, Young Ik Eom

2026. 08. 11

Presentation by Jung In, Minsu Kim

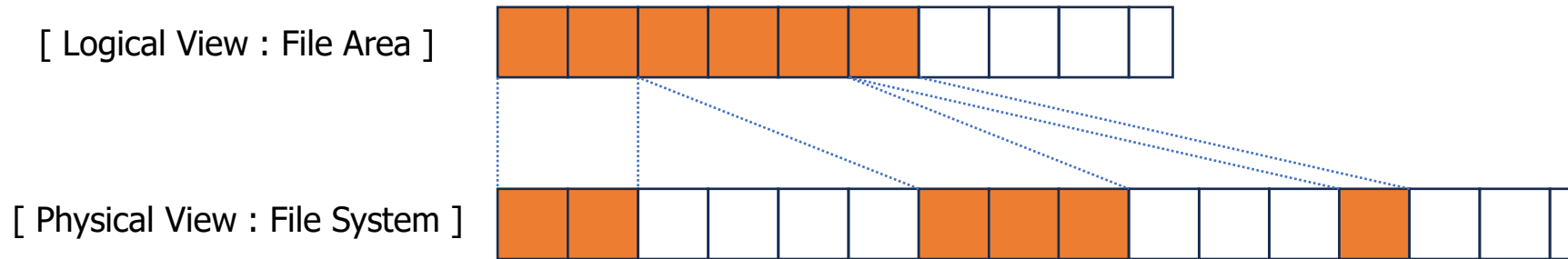
wjddls0318@dankook.ac.kr, minsk0515@dankook.ac.kr

Contents

1. Problem Background
2. Experimental Performance Analysis
3. FragPicker Introduction
4. Evaluation
5. Conclusion

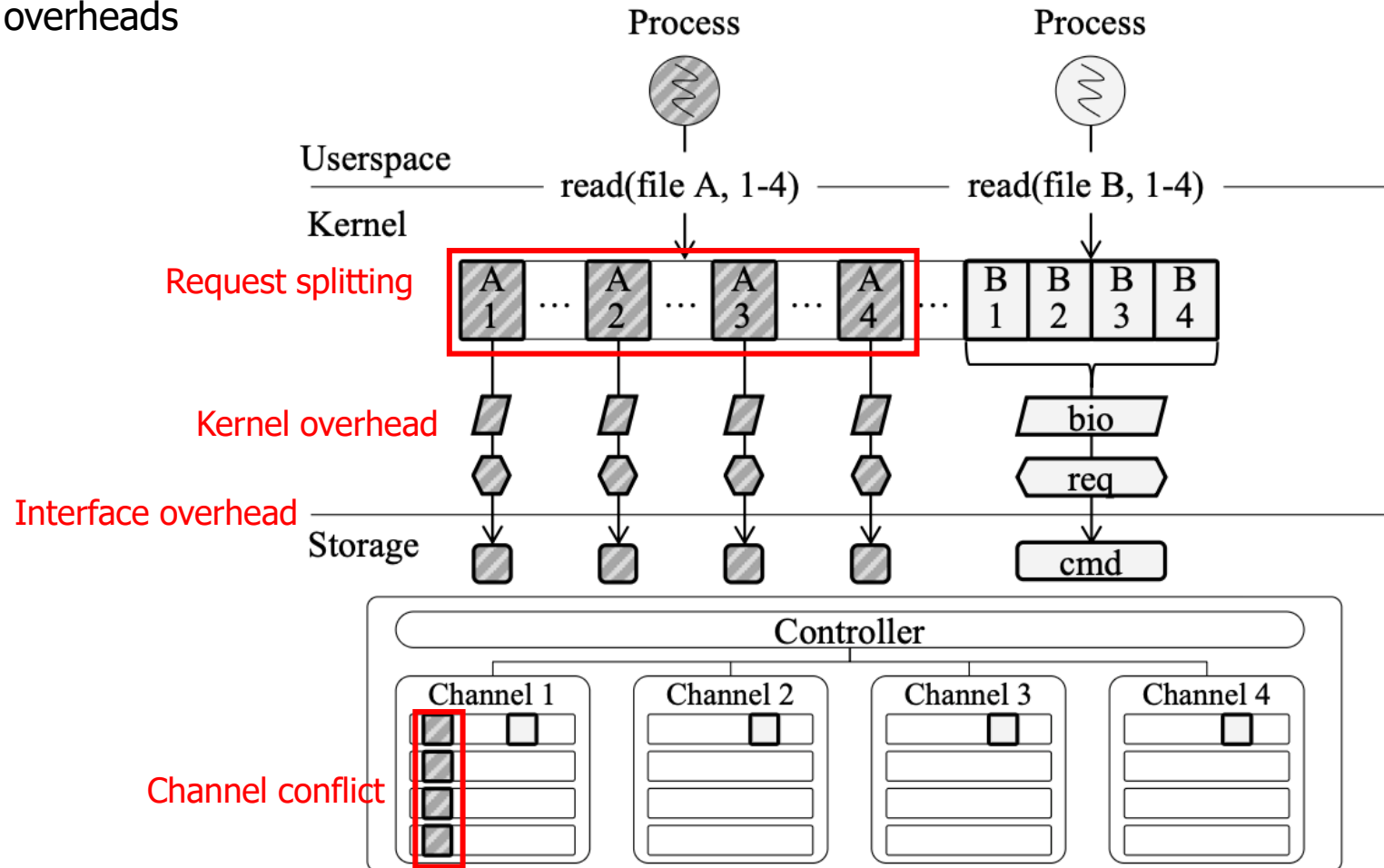
Problem Background - Fragmentation

- What is Fragmentation?



Problem Background - Fragmentation

Request splitting and overheads



Experimental Performance Analysis - metrics

- frag size: fragment의 크기
- frag distance: fragment간 떨어져있는 거리

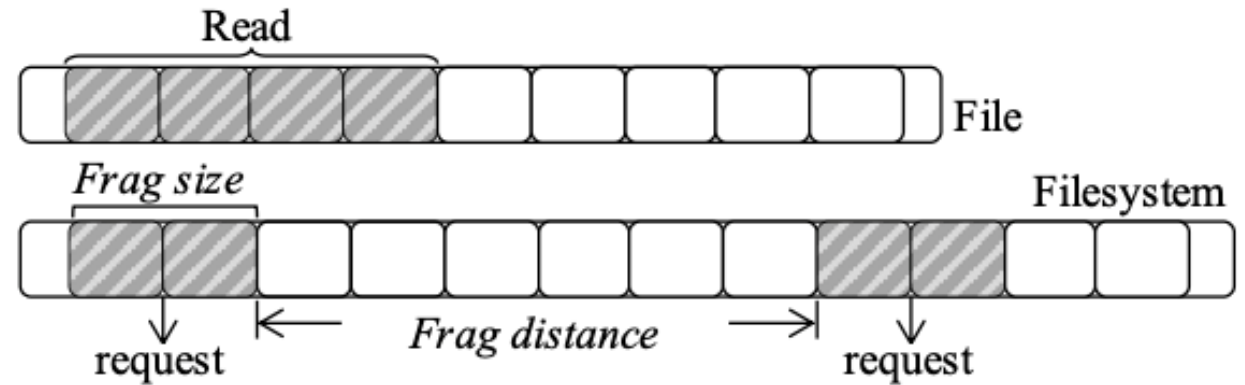
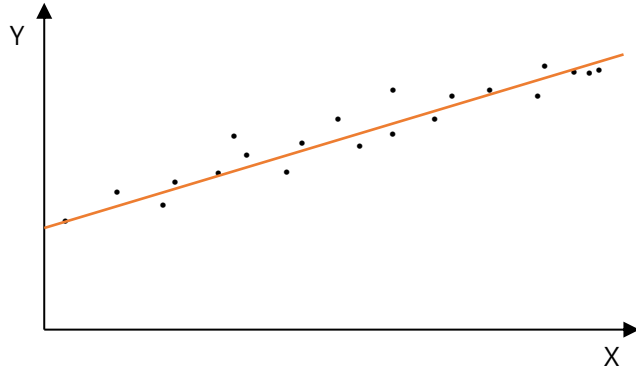


Figure 3. Fragmentation metrics

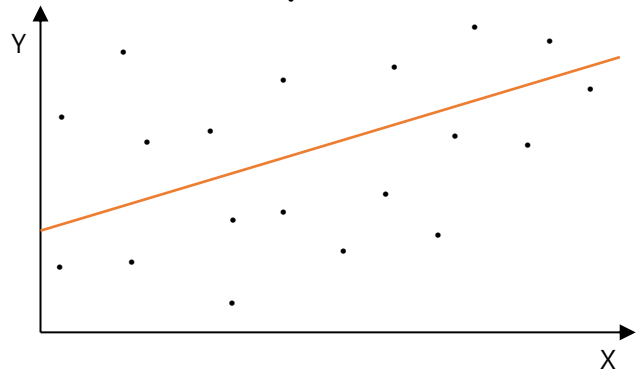
Experimental Performance Analysis – CC



높은 CC값

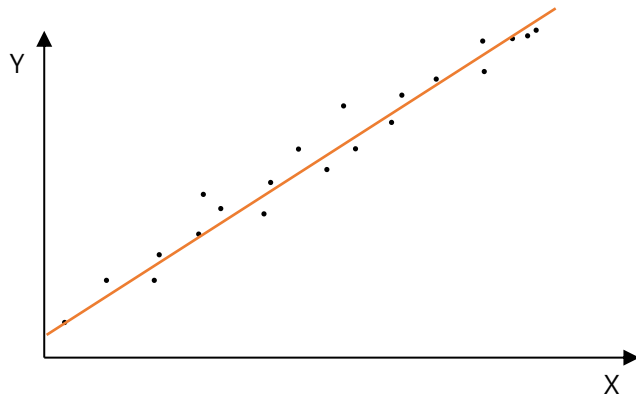
$$CC(X, Y) = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 \sum (y - \bar{y})^2}}$$

- 두 변수 사이의 선형적 관계의 강도, 즉 연관성을 나타냄



낮은 CC값

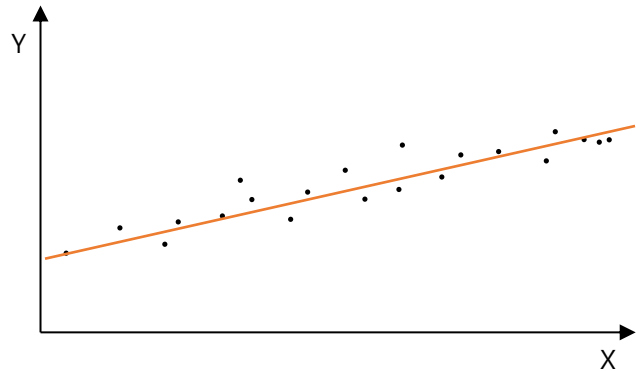
Experimental Performance Analysis – NLRS



높은 NLRS값

$$NLRS(X, Y) = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}$$

- X의 변화에 따라 Y가 얼마나 민감하게 반응하는지 나타냄.
- 기울기로써 나타남



낮은 NLRS값

Experimental Performance Analysis - Result

- 막대: frag distance 고정(1024KB)
- 선: frag size 고정(4KB)
- Request size = 128KB(fixed)

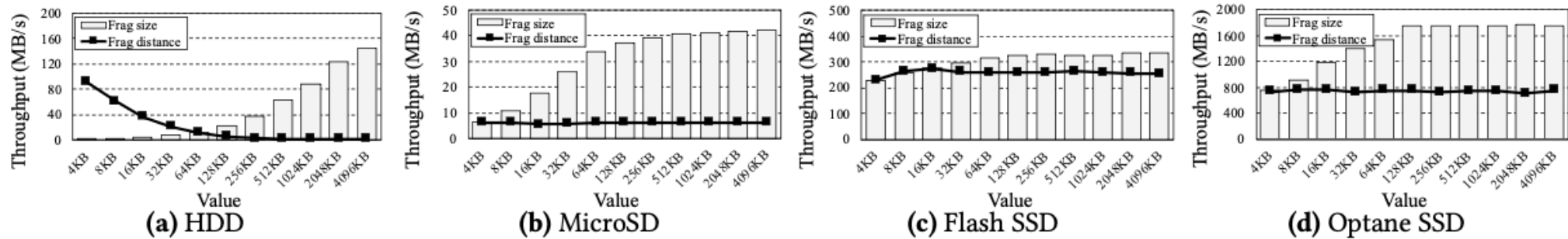


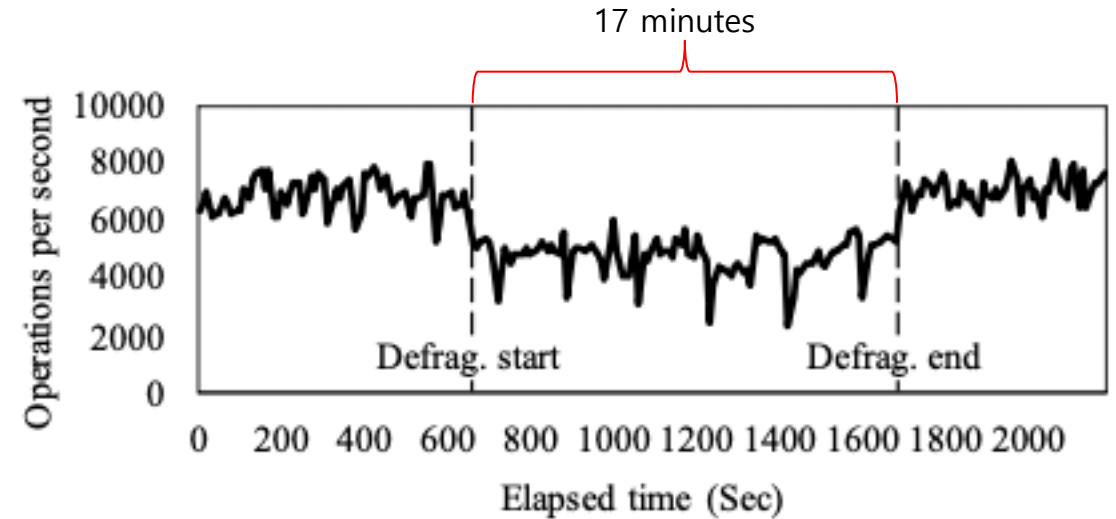
Figure 4. The sequential read performance with various frag size and frag distance

Table 1. CC and NLRs of each metric and the sequential read performance on various storage devices

Metric	Frag Size				Frag Distance	
	Correlation Coefficient (CC)		Normalized Linear Regression Slope (NLRs)		Correlation Coefficient (CC)	Normalized Linear Regression Slope (NLRs)
	Before 128KB	After 128KB	Before 128KB	After 128KB		
HDD	0.9898939	0.9219008	0.1027717	0.0204404	-0.5333534	-0.0345131
MicroSD	0.8889961	0.7488593	0.0380474	0.0001394	0.2562098	0.0000272
Flash SSD	0.8437115	0.7475056	0.0027932	0.0000129	0.3109179	0.0000581
Optane SSD	0.8883913	0.4166667	0.0096721	0.0000022	-0.1539585	-0.0000091

Problem Background – Defragmentation tools

- Ext4 on Flash-SSD
- YCSB workload A with RocksDB
- E4defrag
- 워크로드와 상관없는 파일(30GB) background defragmentation



X축: 경과 시간
Y축: 초당 연산량(YCSB workload A)

- 처리량 평균 32% 감소
- 30GB defragmentation에 17분 소요

FragPicker Overview

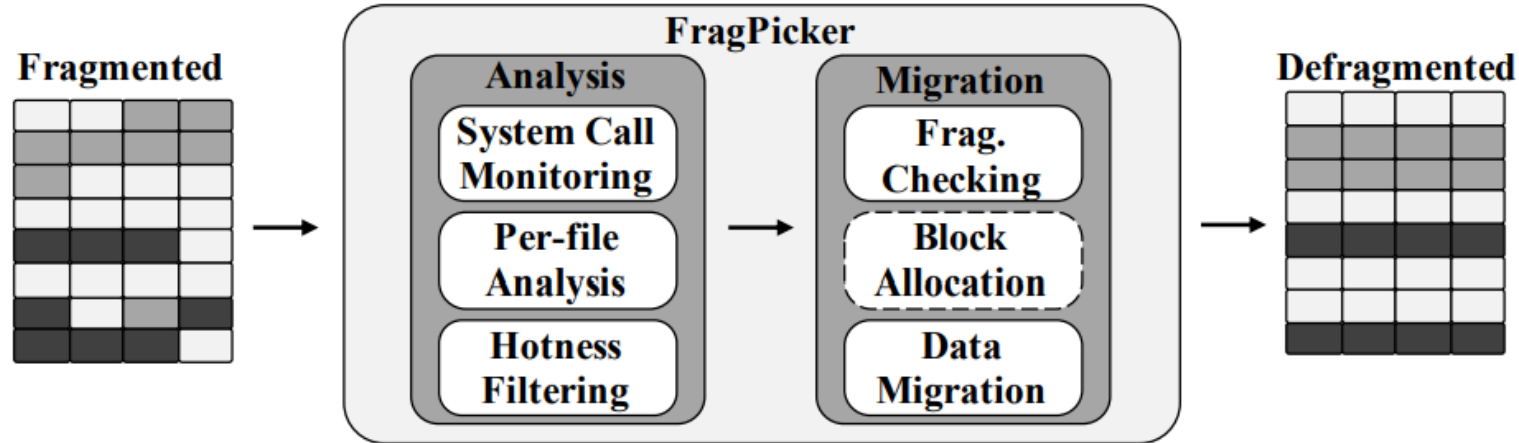


Figure 5. The overview of FragPicker

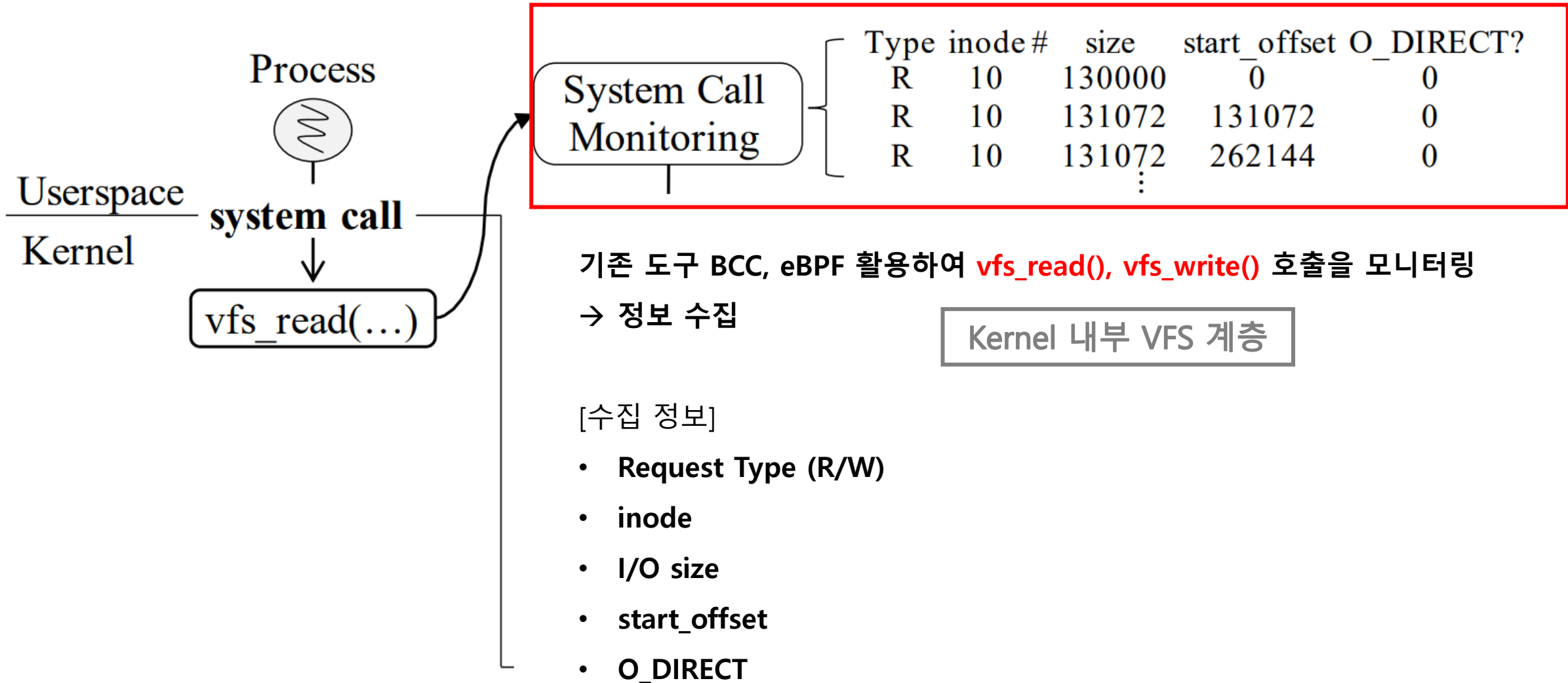
[분석 단계]

System Call 계층에서 I/O 활동 추적
파일 단위로 활동 분석
자주 접근되는 영역만 필터링

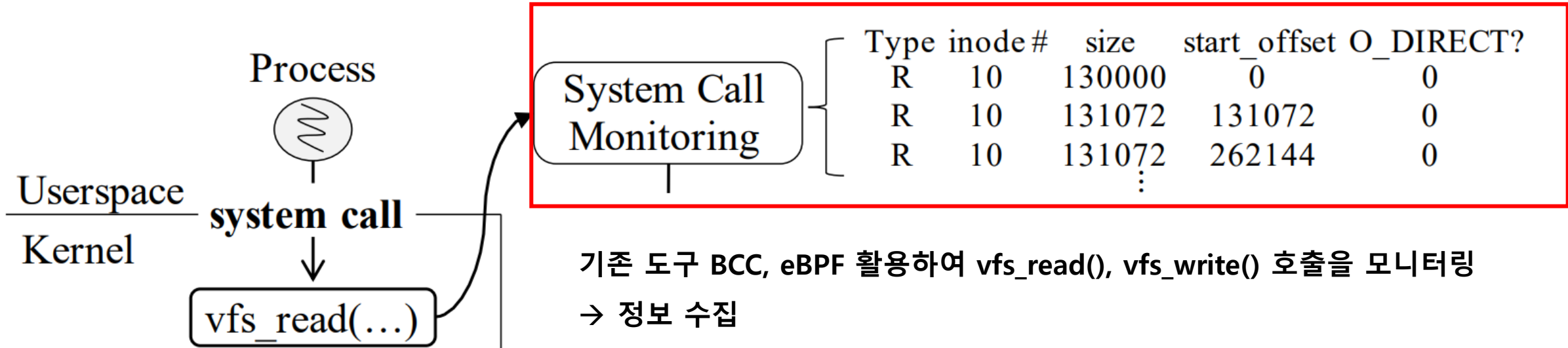
[이주 단계]

실제 조각화 여부 확인
이주할 새로운 Block을 할당
동일 Offset에 Rewrite하여 이주

FragPicker Overview - Analysis



FragPicker Overview - Analysis



기존 도구 BCC, eBPF 활용하여 vfs_read(), vfs_write() 호출을 모니터링
→ 정보 수집

[수집 정보]

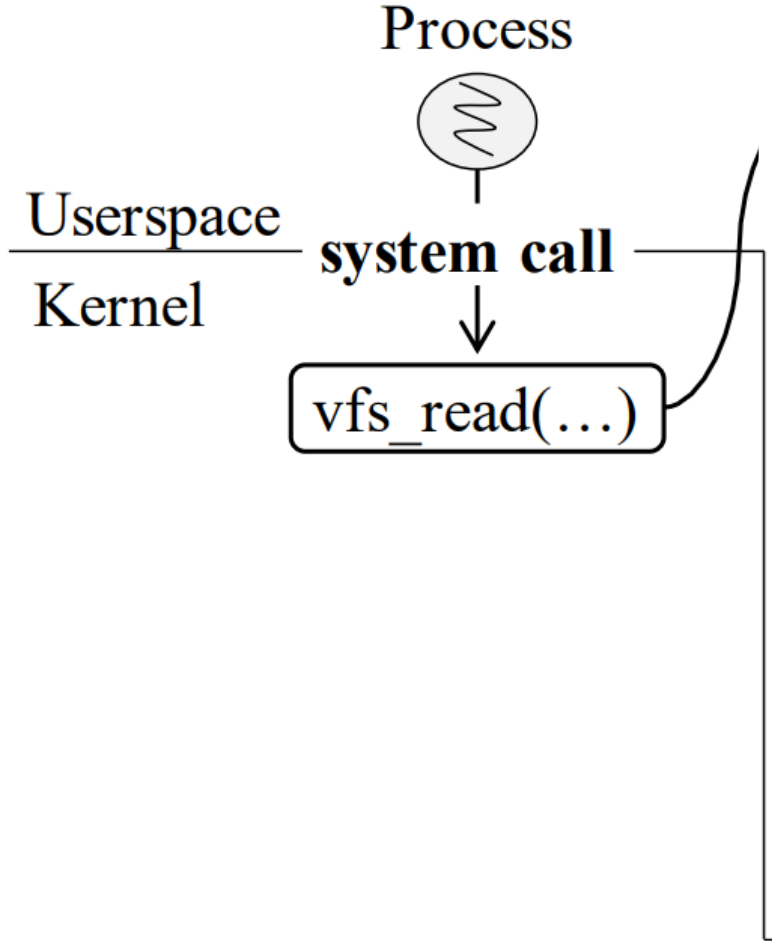
- Request Type (R/W)
- inode
- I/O size
- start_offset
- **O_DIRECT**

Read Type	How Works	O_DIRECT Value
Read-Ahead	Page Cache를 우선 Read	0
Direct I/O	저장장치에서 Direct Read	1

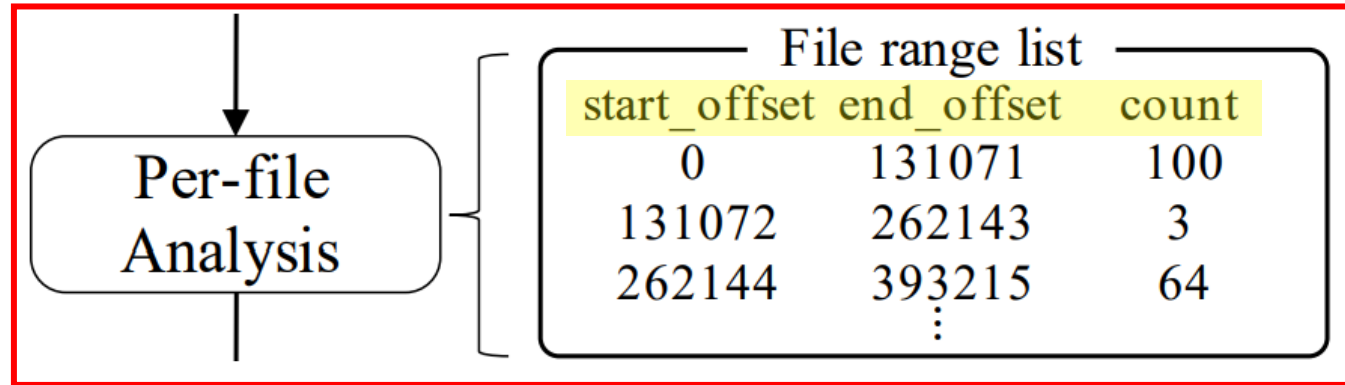
FragPicker는 System Call 계층에서 동작
실제 OS 동작에 가깝게 분석하는 목적

O_DIRECT = 0 && Buffered Sequential Read
→ 파일 분석 시, 분석 범위를 128KB로 확장

FragPicker Overview - Analysis



수집된 I/O 활동 정보를 파일별로 분석 + 각 파일에 대한 파일 범위 목록을 생성



- start_offset 기준 정렬
- 겹치는 범위를 Merge + 범위 내 접근 count
- 독립된 Merge 범위 → File range List 저장

단편화 감소 + Migration I/O 감소

FragPicker Overview - Analysis

Algorithm 1: Merging Overlapped I/Os

EO is end_offset

OW is overlap_window

forall *entry* $\in$ file range list **do**

if *is_overlapped*(*entry*, *OW*) **then**
 count++

if *entry.EO* > *OW.EO* **then**
 OW.EO = *entry.EO*

else

store(*OW*, *count*)

OW = *entry*

- Entry: 파일 접근 범위 하나
- OW(overlap_window): 현재 겹쳐서 병합 중인 범위
- EO(end_offset): 해당 범위의 끝 위치
- count: 해당 범위 내 접근된 횟수

FragPicker Overview - Analysis

Algorithm 1: Merging Overlapped I/Os

EO is end_offset

OW is overlap_window

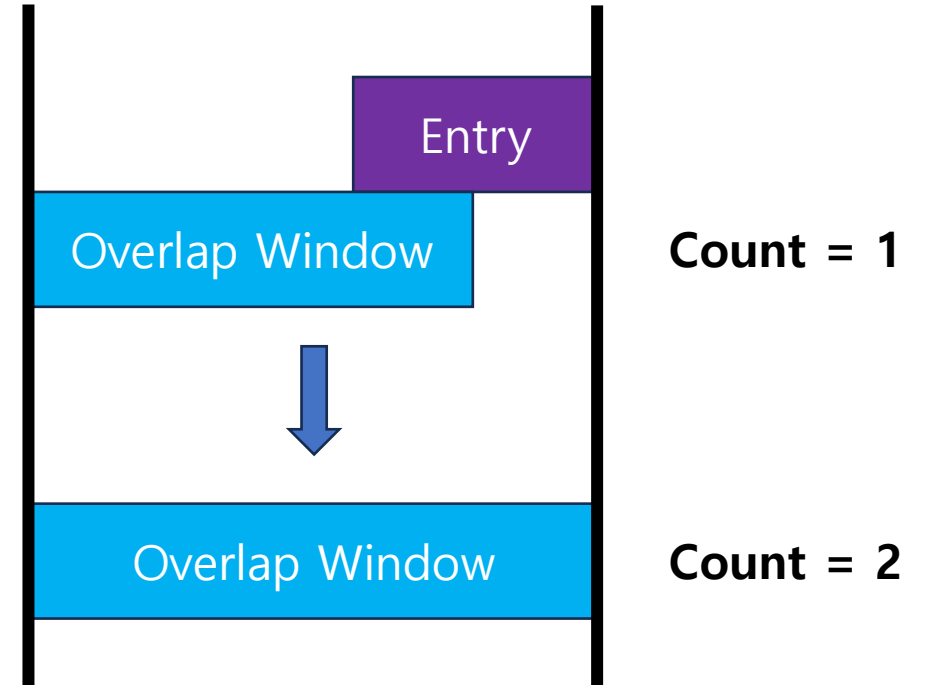
forall *entry* $\in$ file range list **do**

if *is_overlapped*(*entry*, *OW*) **then**
 count++

if *entry.EO* > *OW.EO* **then**
 OW.EO = *entry.EO*

else

store(*OW*, *count*)
 OW = *entry*



Entry: 파일 접근 범위 하나

OW: 현재 겹쳐서 병합 중인 범위

EO: 해당 범위의 끝 위치

count: 해당 범위 내 접근된 횟수

FragPicker Overview - Analysis

Algorithm 1: Merging Overlapped I/Os

EO is end_offset

OW is overlap_window

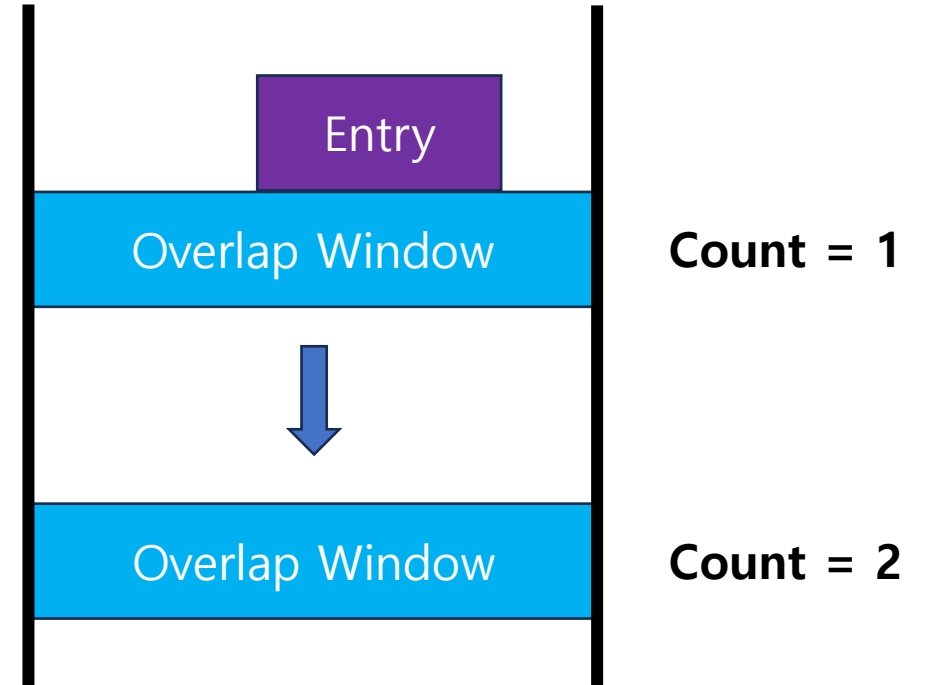
forall *entry* $\in$ file range list **do**

if *is_overlapped*(*entry*, *OW*) **then**
 count++

if *entry.EO* > *OW.EO* **then**
 OW.EO = *entry.EO*

else

store(*OW*, *count*)
 OW = *entry*



Entry: 파일 접근 범위 하나
OW: 현재 겹쳐서 병합 중인 범위
EO: 해당 범위의 끝 위치
count: 해당 범위 내 접근된 횟수

FragPicker Overview - Analysis

Algorithm 1: Merging Overlapped I/Os

EO is end_offset

OW is overlap_window

forall *entry* $\in$ file range list **do**

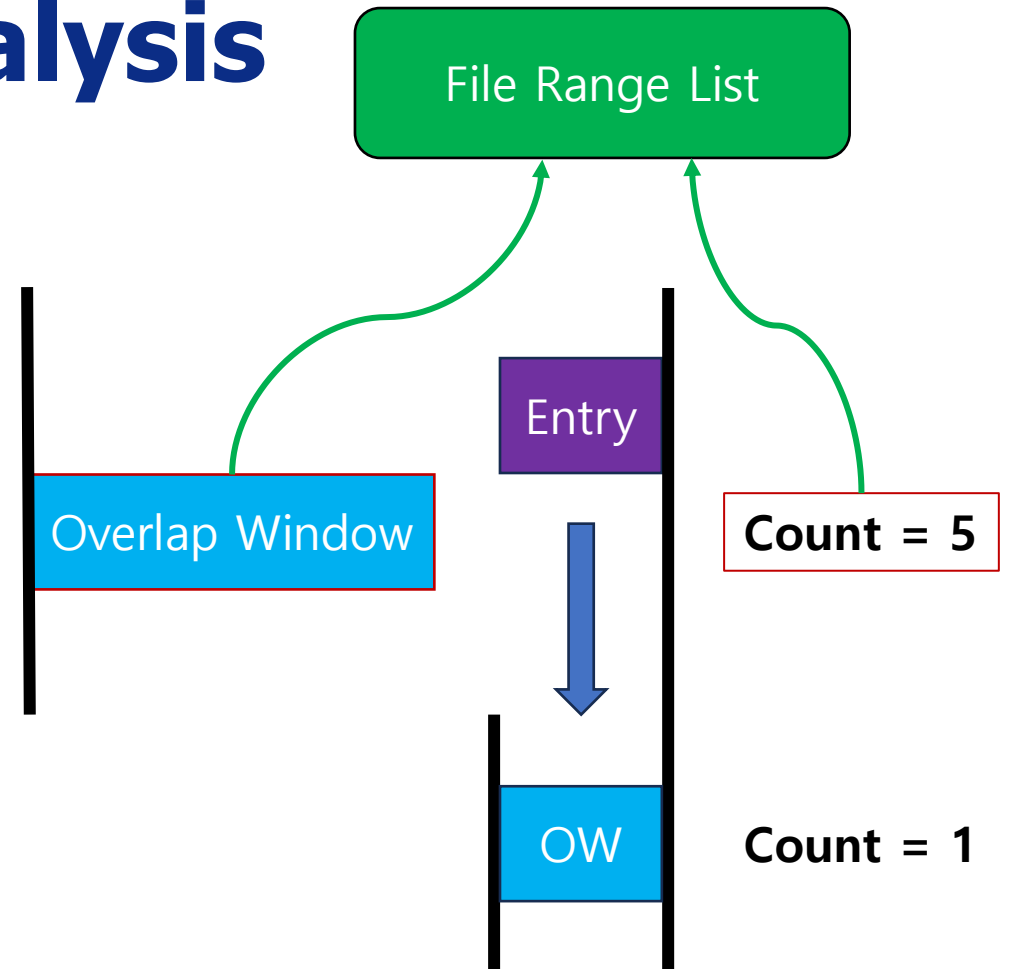
if *is_overlapped*(*entry*, *OW*) **then**
 count++

if *entry.EO* > *OW.EO* **then**
 OW.EO = *entry.EO*

else

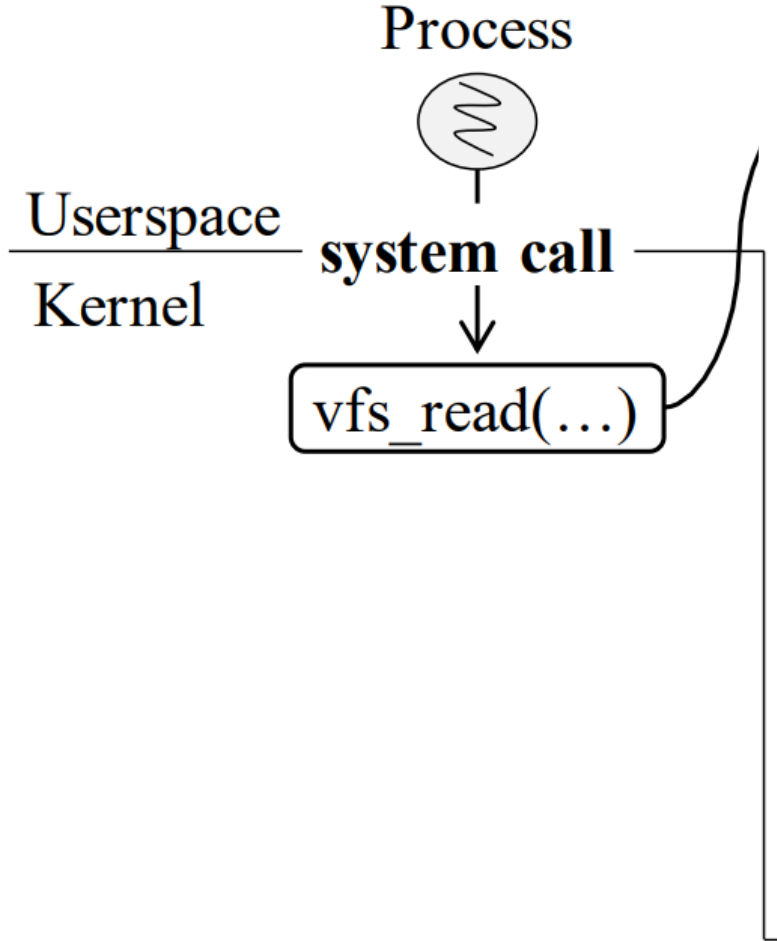
store(*OW*, *count*)

OW = *entry*

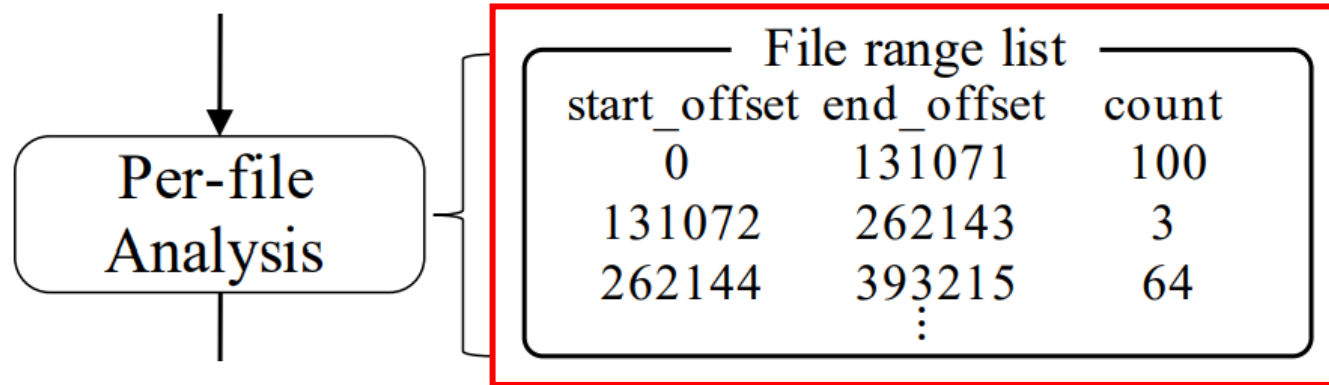


Entry: 파일 접근 범위 하나
OW: 현재 겹쳐서 병합 중인 범위
EO: 해당 범위의 끝 위치
count: 해당 범위 내 접근된 횟수

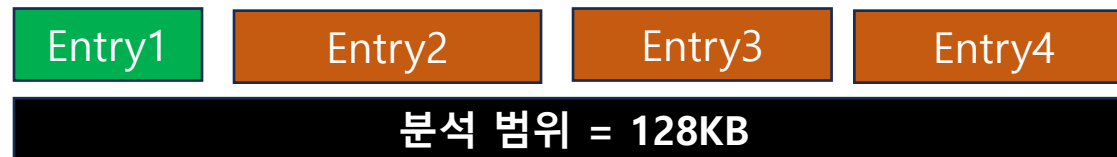
FragPicker Overview - Analysis



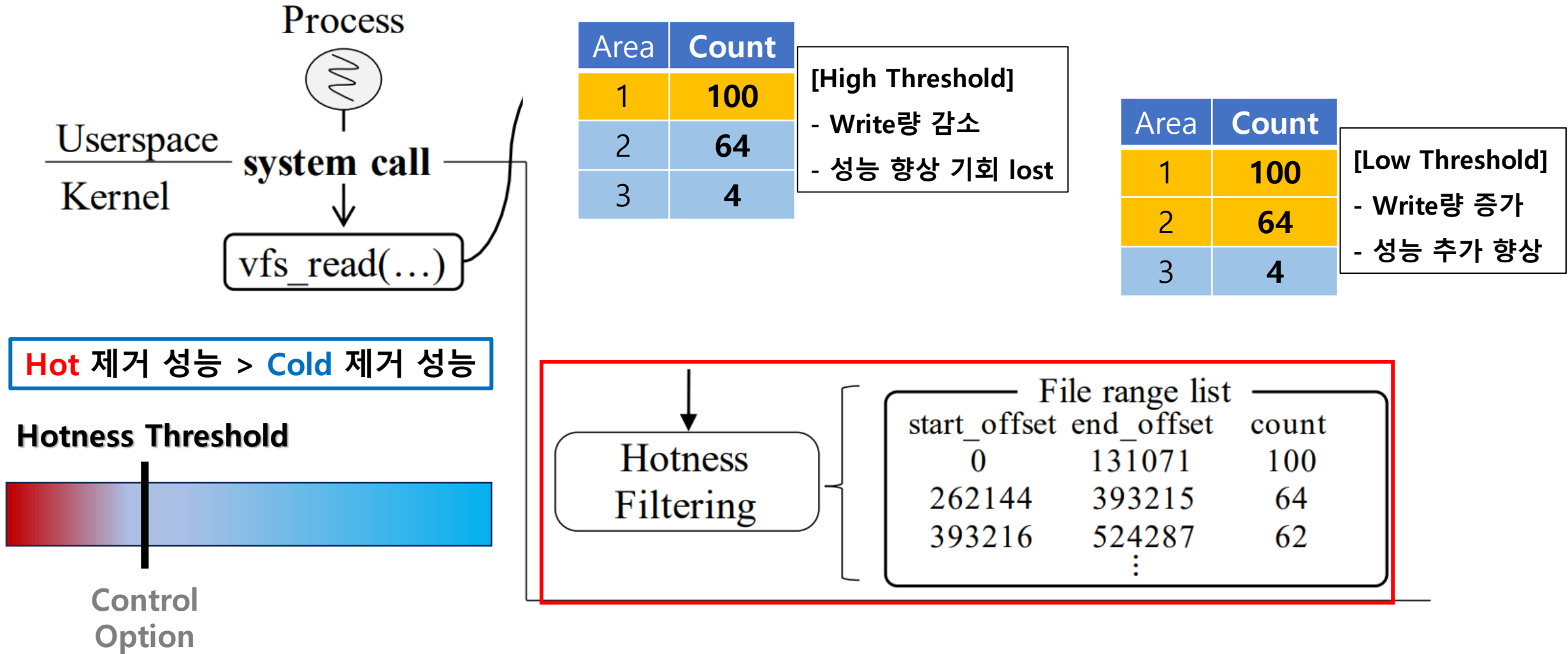
수집된 I/O 활동 정보를 파일별로 분석 + 각 파일에 대한 파일 범위 목록을 생성



- FS Block 경계 고려 offset 조정 (Data 손실 방지)
- Buffered 순차 읽기의 경우 Read-Ahead 반영 + 후속 요청 무시



FragPicker Overview - Analysis



FragPicker Overview

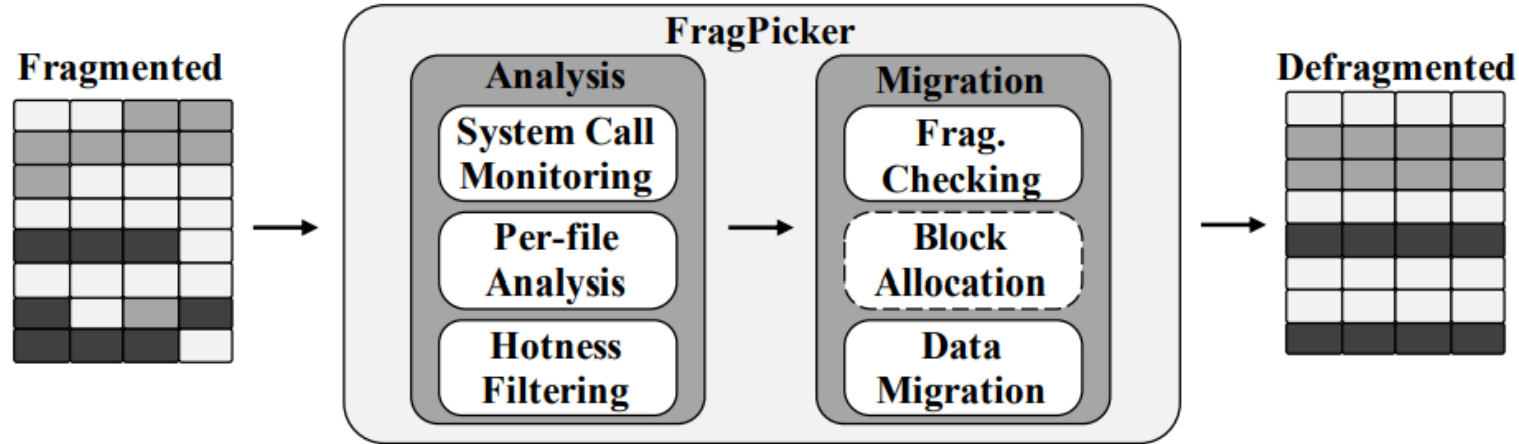


Figure 5. The overview of FragPicker

[분석 단계]

System Call 계층에서 I/O 활동 추적

파일 단위로 활동 분석

자주 접근되는 영역만 필터링

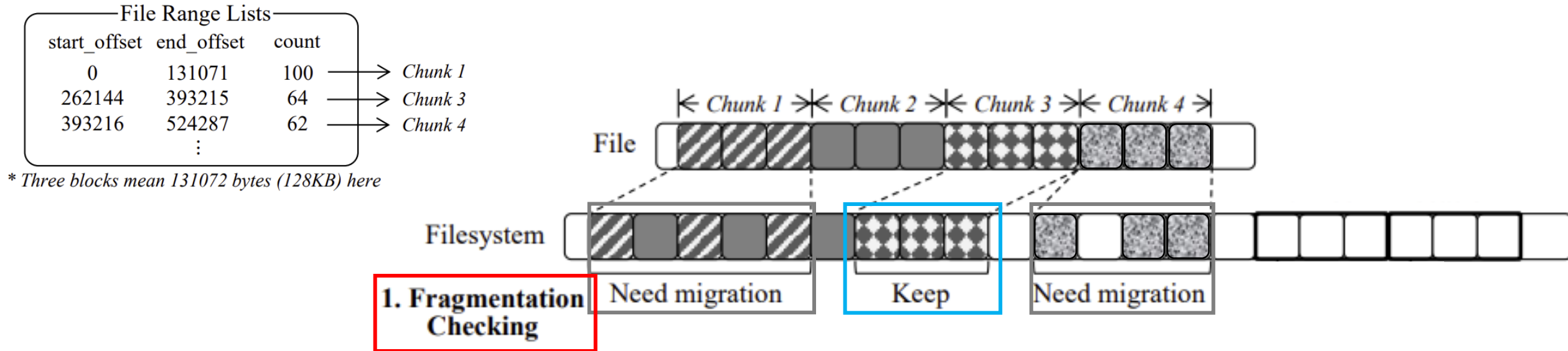
[이주 단계]

실제 조각화 여부 확인

이주할 새로운 Block을 할당

동일 Offset에 Rewrite하여 이주

FragPicker Overview - Migration

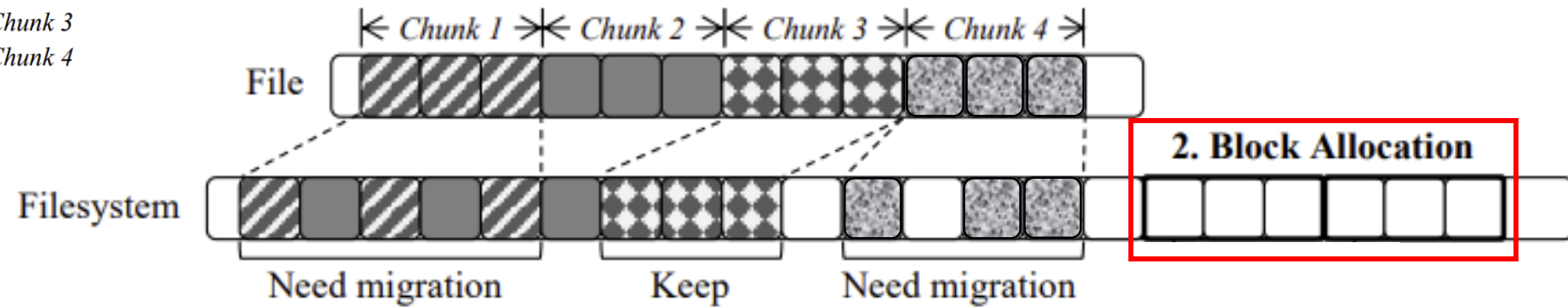


Filefrag 프로그램 사용

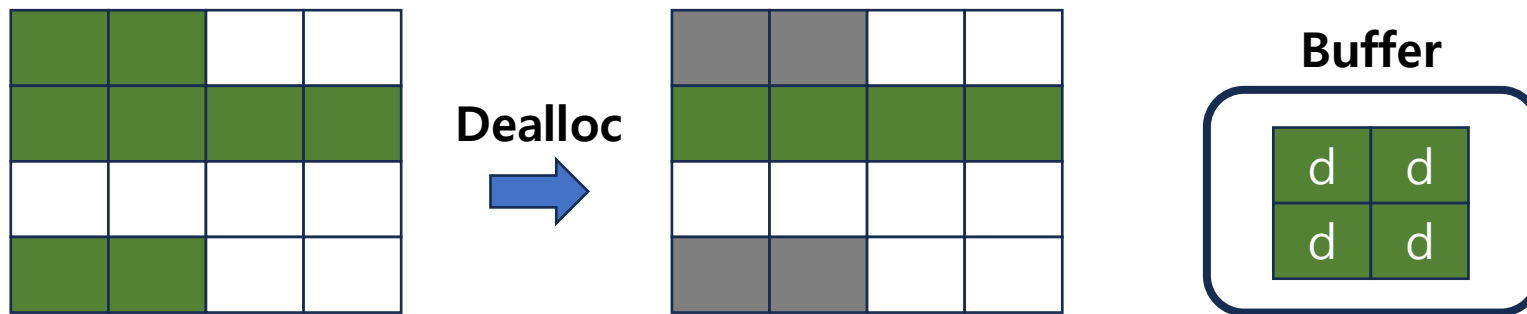
FragPicker Overview - Migration

File Range Lists			
start_offset	end_offset	count	
0	131071	100	→ <i>Chunk 1</i>
262144	393215	64	→ <i>Chunk 3</i>
393216	524287	62	→ <i>Chunk 4</i>
⋮			

* Three blocks mean 131072 bytes (128KB) here



In-Place Update (ex. Ext4..)

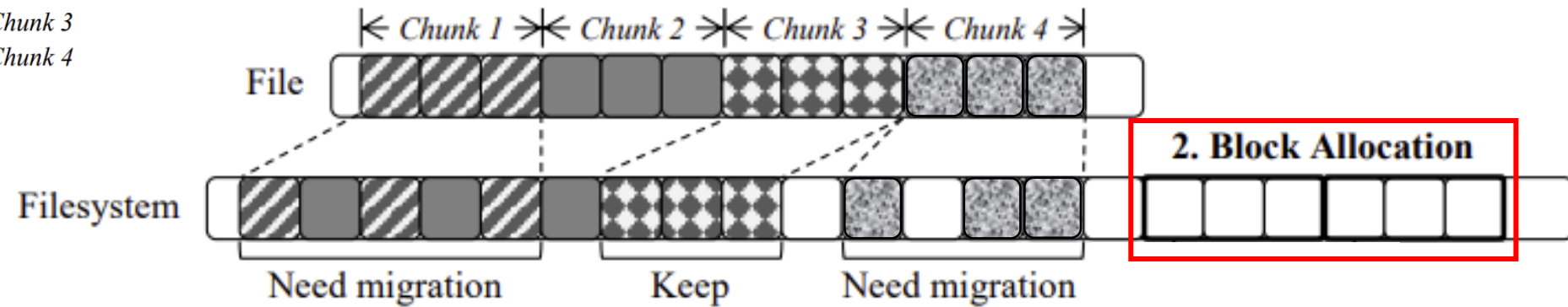


Using `fallocate()`

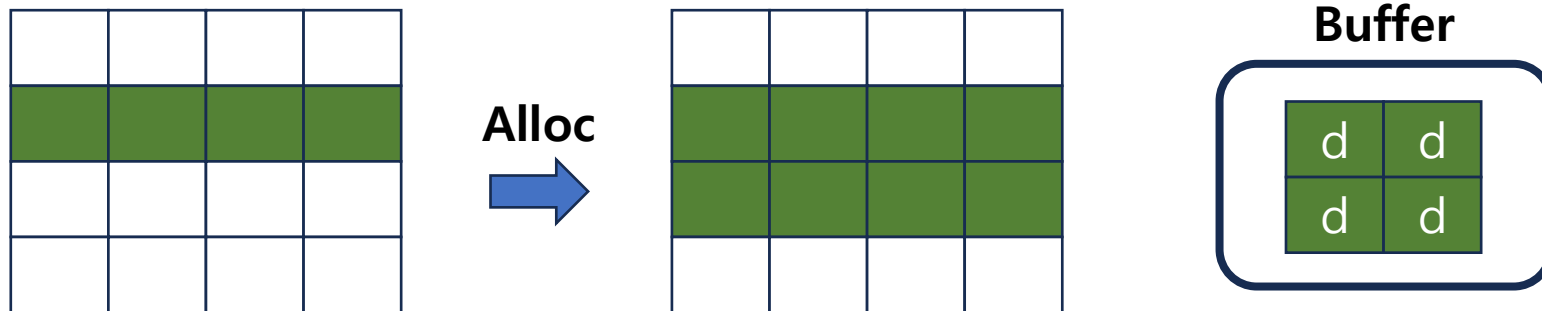
FragPicker Overview - Migration

File Range Lists			
start_offset	end_offset	count	
0	131071	100	→ <i>Chunk 1</i>
262144	393215	64	→ <i>Chunk 3</i>
393216	524287	62	→ <i>Chunk 4</i>
⋮			

* Three blocks mean 131072 bytes (128KB) here



In-Place Update (ex. Ext4..)

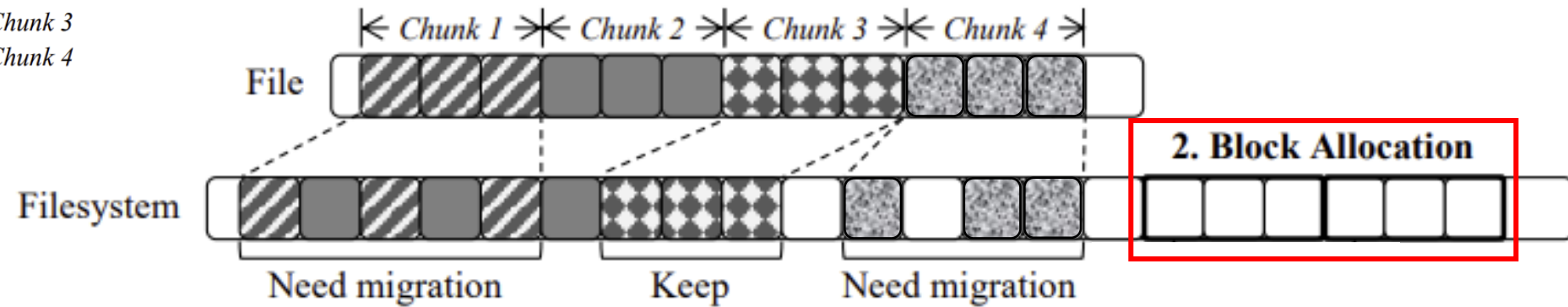


Using `fallocate()`

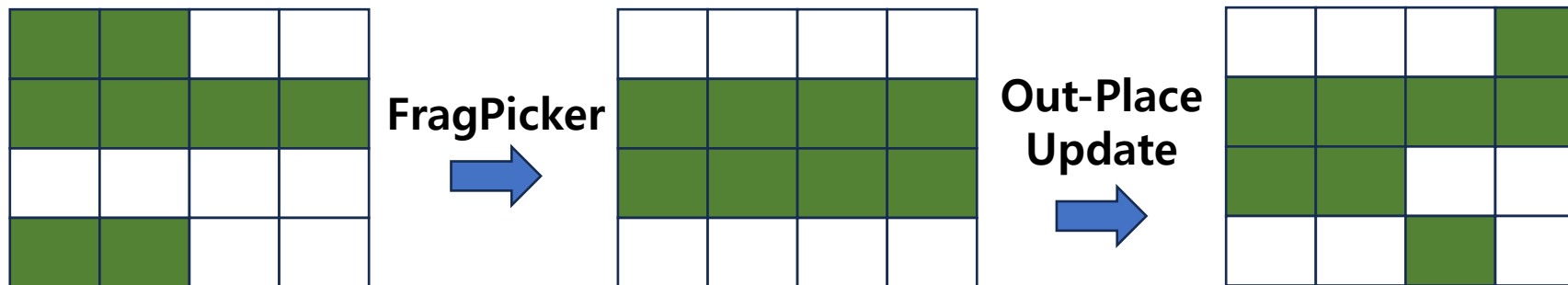
FragPicker Overview - Migration

File Range Lists			
start_offset	end_offset	count	
0	131071	100	→ <i>Chunk 1</i>
262144	393215	64	→ <i>Chunk 3</i>
393216	524287	62	→ <i>Chunk 4</i>
⋮			

* Three blocks mean 131072 bytes (128KB) here



Out-Place Update (ex. Btrfs..)



FragPicker 의미 X
→ Block Allocation
스킵 가능

FragPicker Overview - Migration

In-Place Out-Place

Analysis

Check

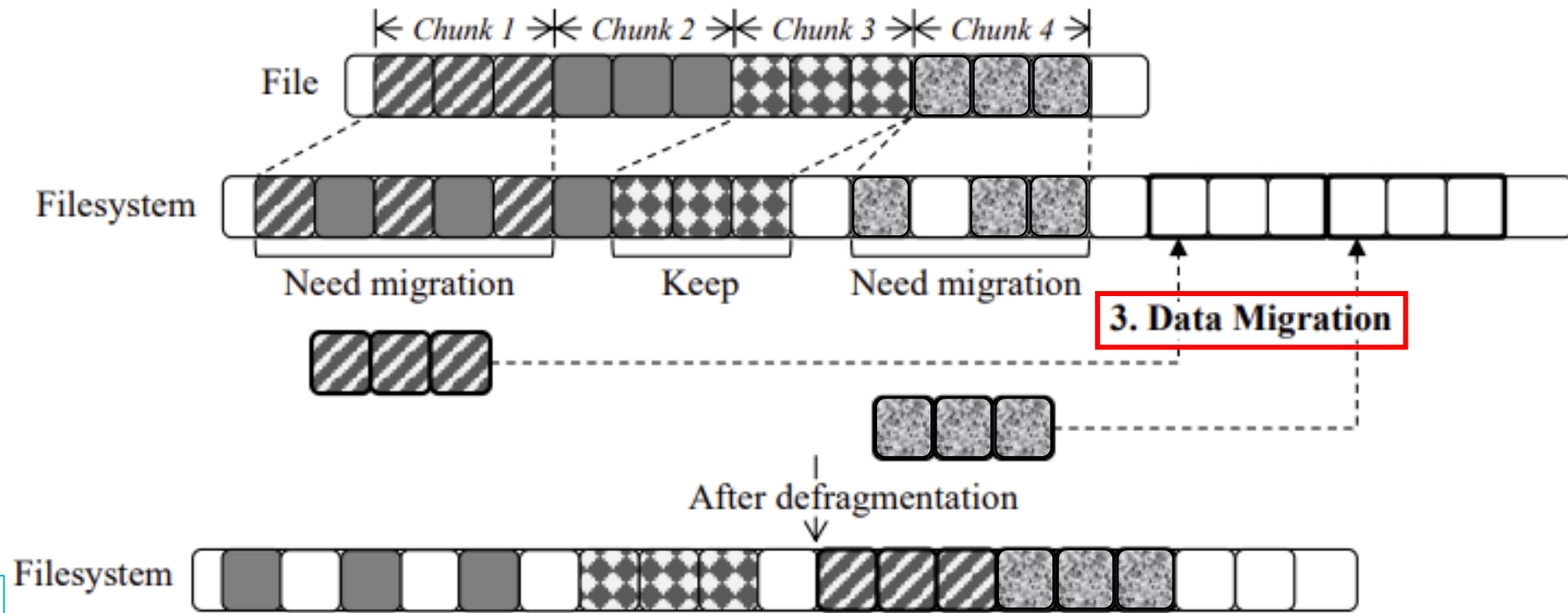
Allocation

Migration

FragPicker가
할당한 Block에
Rewrite

Update 하면서
할당한 Block에
Rewrite

데이터를 동일한 Offset에 Rewrite



Hot + 조각화 Block만 처리 → Write 최소화

청크 간 거리가 증가 → 현대 저장장치에서는 성능 좌우하는 핵심 요소 X

Evaluation – Experimental Setup

- F2FS: 다른 파일 시스템 defrag tool 모방
- YCSB 실험: defrag 하는 도중 성능 측정
- 그 외: defrag 후에 워크로드의 성능을 측정
- FragPick: migration 전에 동일 workload를 실행함
- analysis를 수행하기 위함이며, YSCB 실험은 예외

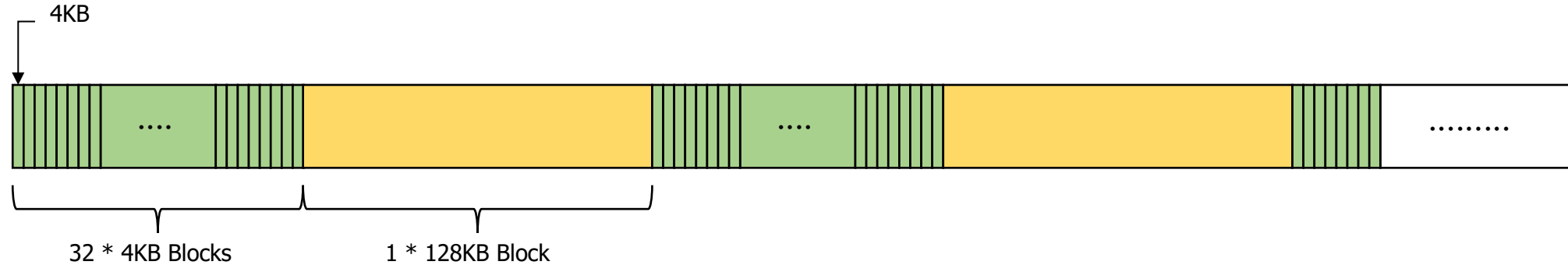
Table 2. Experimental setup

CPU	Intel Xeon E5-2620 v4 2.1GHz 8 core
Memory	DDR4 2133MHz 16GB
Storage	Samsung HDD 7200RPM 1TB Samsung MicroSD card EVO type A1 128GB Samsung SATA Flash SSD 850 PRO 256GB Intel NVMe Optane SSD 905P 960GB
OS	Ubuntu 18.04 LTS with Linux Kernel 5.7.0

Evaluation – Synthetic Workloads

- Synthetic Fragmented File Layout:

$32 * 4\text{KB Blocks} + 128\text{KB Block} + 32 * 4\text{KB Blocks} + \dots$



- Request size = 128KB
- Skip/Gap = 288KB

Evaluation – Synthetic Workloads

FragPicker-B : analysis phase 우회

Conv. : Conventional scheme

Btrfs의 Conv.-T는 자체 최적화 방법을 적용한 것

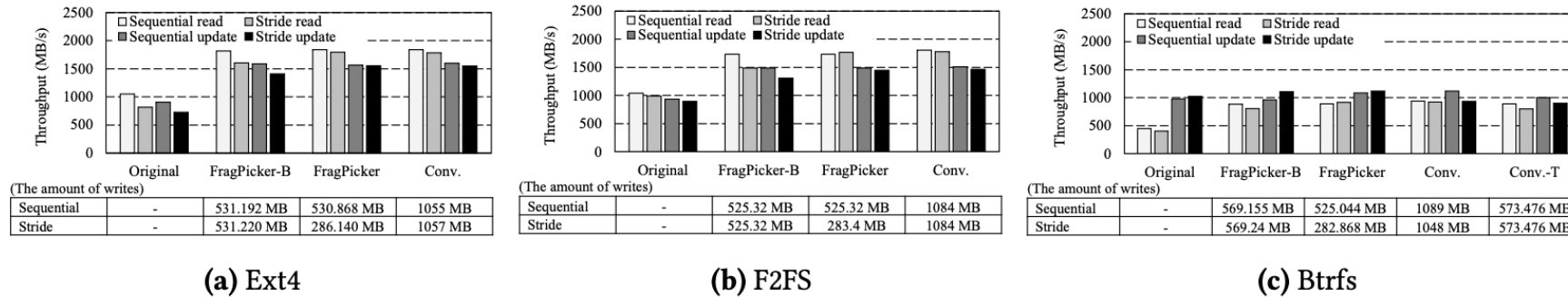


Figure 8. The experimental results of the synthetic workloads on the Optane SSD (1GB file)

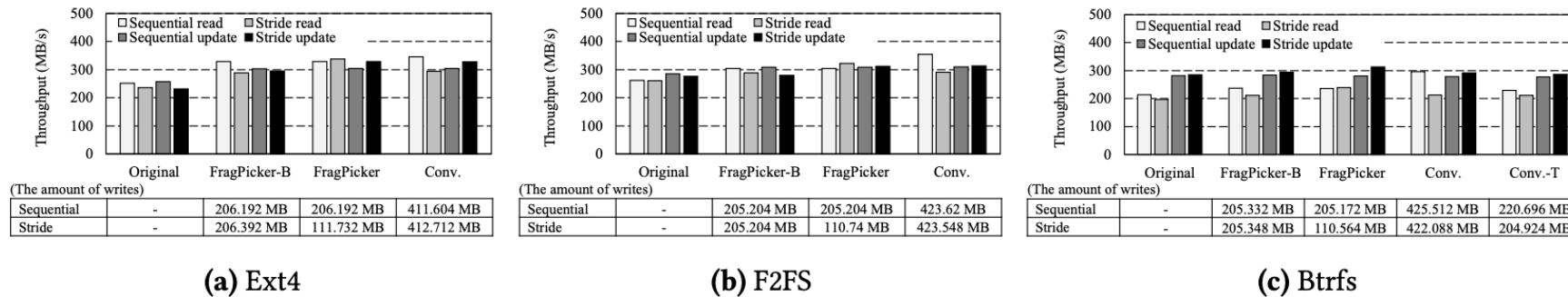
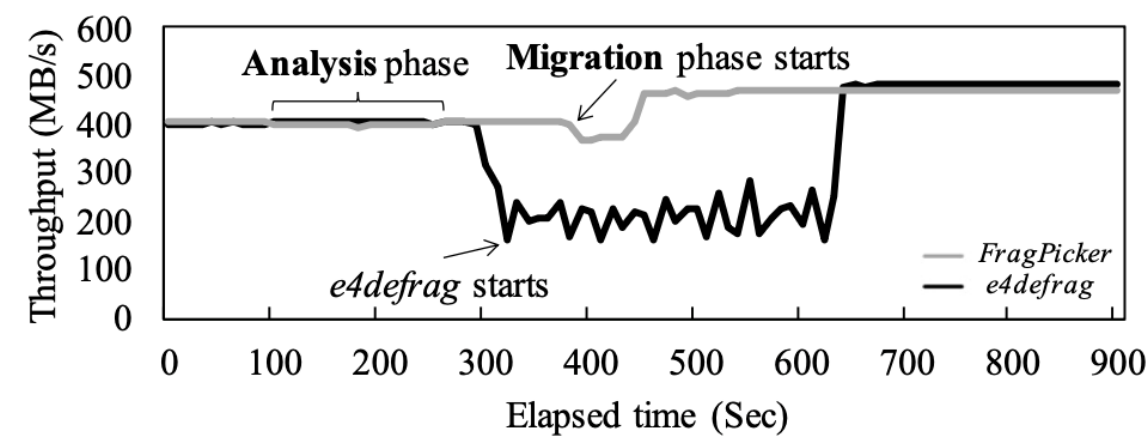


Figure 9. The experimental results of the synthetic workloads on the Flash SSD (400MB file)

Hotness Filtering과 Fragmentation Checking으로 인한 write overhead 감소 확인 가능

Evaluation – Database Workload

- Ext4, Optane SSD
- YCSB workload-C(Read 100%, zipfian distributution)



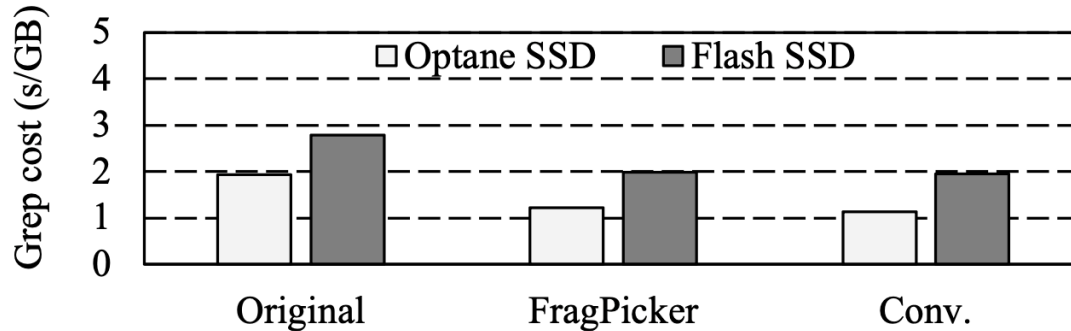
	FragPicker	e4defrag
Defrag중 Throughput 감소율	1.4%(analysis) 7.4%(migration)	47%
소요 시간	54s(migration)	331s

Figure 10. YCSB workload-C with RocksDB on Ext4

SQLite/Btrfs/MicroSD에서도 conventional과 유사한 성능 향상 + defrag overhead 감소 확인

Evaluation – Fileserver Workload

- Filebench fileserver workload
- $\text{grep cost} = [\text{elapsed time}] / [\text{전체 fileserver data size}]$
- 작을수록 성능이 높음을 나타냄(s/GB)



(The amount of writes)

Optane SSD	-	27653 MB	49118 MB
Flash SSD	-	10862 MB	24597 MB

Figure 11. The Filebench fileserver workload on F2FS

- Conventional scheme과 유사한 성능, 적은 write overhead
- Grep cost: Original 대비 37% 감소
- Write amount: Conventional 대비 44% 감소
- 파일당 평균 파편 갯수 1395 -> 1.77 (Optane)

Filebench fileserver workload 실행

fileserver 데이터 구성 및
fragmentation 상태 형성

Original
(그대로)

Defragmentation

FragPicker

Conv.

recursive grep

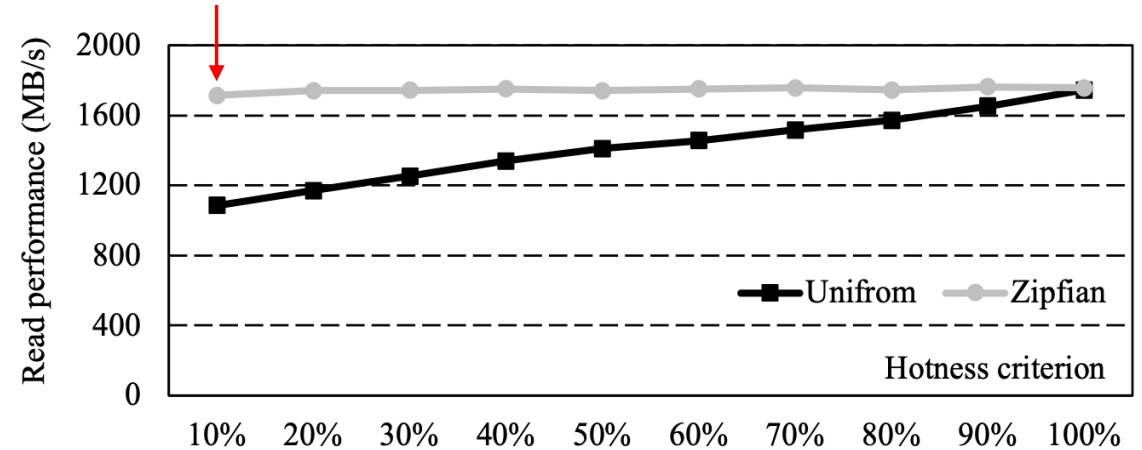
grep elapsed time

grep cost (s/GB)

Evaluation – Hotness Filtering test

- Optane SSD
- Same fragmented file as the synthetic experiment(1GB)
- Hotness criterion = 전체 data중 migration 후보로 삼는 비율
- Zipfian distribution: 상위 순위 데이터에 접근이 집중되는 분포

Zipfian: 10% hot data만 migration해도 높은 성능 유지



(The amount of writes)

Unifrom (MB)	32	66	97	127	164	198	231	257	301	337
Zipfian (MB)	0.7	1	2.4	2.1	2.6	4.5	5.4	6.1	7.2	7.5

Figure 12. The hotness test under the Unif. and Zipf. dist.

Uniform: Hotness criterion 완화에 따라 성능과 write amount 증가
 Zipfian: 성능 변화 크게 없음, write amount 증가

Limitations

Page Cache 영향 미반영

System call 계층에서 I/O를 모니터링하기에 page cache 영향이 전부 고려되지 않을 수 있음

PBA 레벨 단편화 미탐지

LBA -> PBA간 단편화로 인한 resource conflict는 발생 가능

Frag distance 미고려

HDD 등 seek time이 있는 장치에는 부적합

Conclusion

Observation

- 현대 저장장치에서 fragmentation 성능 저하의 주요 원인 = Request Splitting
- Fragment distance 자체는 상대적으로 중요하지 않음

Key Idea

- Application I/O 분석
- 성능에 중요한 fragmented block만 선택하여 migration

Effect

- I/O 성능 ↑
- Defragmentation write ↓

Advantage

- Filesystem에 의존하지 않음
- Kernel modification 없음

Thank you